

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра компьютерных и информационных наук

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 2

дисциплина: Архитектура компьютеров

Студент: Симакова Алина

Группа: НКАбд-05-25

МОСКВА

2025 г.

Оглавление

1 Цель работы.....	3
2 Задание.....	4
3 Теоретическое введение.....	5
4 Выполнение лабораторной работы.....	6
4.1 Реализация подпрограмм в NASM.....	6
4.2 Отладка программ с помощью GDB.....	9
4.3 Добавление точек останова.....	13
4.4 Работа с данным программ в GDB.....	15
4.5 Обработка аргументов командной строки в GDB.....	19
5 Задания для самостоятельной работы.....	20
6 Выводы.....	25
Список литературы.....	26

1 Цель работы

Целью данной работы является приобретение навыков написания программ с использованием подпрограмм, знакомство с методами отладки при помощи GBD и его основными возможностями.

2 Задание

На основе методических указаний освоить реализацию подпрограмм в NASM, осуществить отладку программ с помощью GDB и выполнить задания по материалам лабораторной работы.

3 Теоретическое введение

Отладка — это процесс поиска и исправления ошибок в программе. В общем случае его можно разделить на четыре этапа:

- обнаружение ошибки;
- поиск её местонахождения;
- определение причины ошибки;
- исправление ошибки.

Можно выделить следующие типы ошибок:

- синтаксические ошибки — обнаруживаются во время трансляции исходного кода и вызваны нарушением ожидаемой формы или структуры языка;
- семантические ошибки — являются логическими и приводят к тому, что программа запускается, отработывает, но не даёт желаемого результата;
- ошибки в процессе выполнения — не обнаруживаются при трансляции и вызывают прерывание выполнения программы (например, это ошибки, связанные с переполнением или делением на ноль).

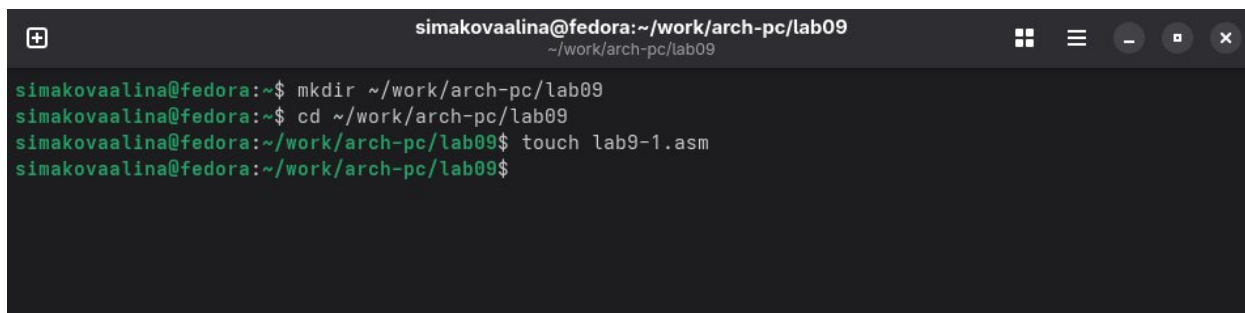
Второй этап — поиск местонахождения ошибки. Некоторые ошибки обнаружить довольно трудно. Лучший способ найти место в программе, где находится ошибка, это разбить программу на части и произвести их отладку отдельно друг от друга.

Третий этап — выяснение причины ошибки. После определения местонахождения ошибки обычно проще определить причину неправильной работы программы. Последний этап — исправление ошибки. После этого при повторном запуске программы, может обнаружиться следующая ошибка, и процесс отладки начнётся заново.

4 Выполнение лабораторной работы

4.1 Реализация подпрограмм в NASM

В домашней директории создаю каталог для выполнения лабораторной работы №9 (рис. 4.1).



```
simakovaalina@fedora:~/work/arch-pc/lab09
simakovaalina@fedora:~$ mkdir ~/work/arch-pc/lab09
simakovaalina@fedora:~$ cd ~/work/arch-pc/lab09
simakovaalina@fedora:~/work/arch-pc/lab09$ touch lab9-1.asm
simakovaalina@fedora:~/work/arch-pc/lab09$
```

Рис. 4.1: Создание рабочего каталога

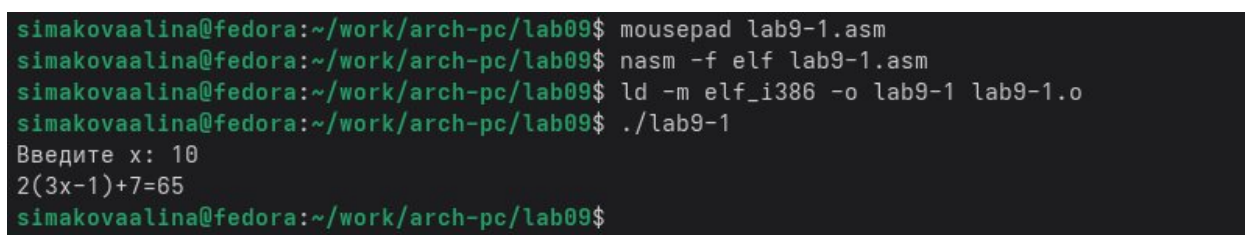
Затем копирую в файл код из листинга, компилирую и запускаю его. Данная программа выполняет вычисление функции (рис. 4.2).



```
simakovaalina@fedora:~/work/arch-pc/lab09$ mousepad lab9-1.asm
simakovaalina@fedora:~/work/arch-pc/lab09$ nasm -f elf lab9-1.asm
simakovaalina@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab9-1 lab9-1.o
simakovaalina@fedora:~/work/arch-pc/lab09$ ./lab9-1
Введите x: 10
2x+7=27
simakovaalina@fedora:~/work/arch-pc/lab09$
```

Рис. 4.2: Запуск программы из листинга

Изменяю текст программы, добавив в неё подпрограмму. Теперь она вычисляет значение функции для выражения $f(g(x))$ (рис. 4.3).



```
simakovaalina@fedora:~/work/arch-pc/lab09$ mousepad lab9-1.asm
simakovaalina@fedora:~/work/arch-pc/lab09$ nasm -f elf lab9-1.asm
simakovaalina@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab9-1 lab9-1.o
simakovaalina@fedora:~/work/arch-pc/lab09$ ./lab9-1
Введите x: 10
2(3x-1)+7=65
simakovaalina@fedora:~/work/arch-pc/lab09$
```

Рис. 4.3: Изменение программы первого листинга

Прикрепляю код программы:

```
%include 'in_out.asm'
SECTION .data
msg: DB 'Введите x: ', 0
result: DB '2(3x-1)+7=', 0
```

SECTION .bss

x: RESB 80

res: RESB 80

SECTION .text

GLOBAL _start

_start:

mov eax, msg

call sprint

mov ecx, x

mov edx, 80

call sread

mov eax, x

call atoi

call _calcul

mov eax, result

call sprint

mov eax, [res]

call iprintLF

call quit

_calcul:

push eax

call _subcalcul

mov ebx, 2

mul ebx

```
add eax, 7
```

```
mov [res], eax
```

```
pop eax
```

```
ret
```

```
_subcalcul:
```

```
mov ebx, 3
```

```
mul ebx
```

```
sub eax, 1
```

```
ret
```

4.2 Отладка программ с помощью GDB

В созданный файл копирую программу второго листинга, транслирую с созданием файла и отладки, после чего компоную и запускаю в отладчике (рис. 4.4).

```
simakovaalina@fedora:~/work/arch-pc/lab09$ touch lab9-2.asm
simakovaalina@fedora:~/work/arch-pc/lab09$ mousepad lab9-2.asm
simakovaalina@fedora:~/work/arch-pc/lab09$ nasm -f elf -g -l lab9-2.lst lab9-2.asm
simakovaalina@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab9-2 lab9-2.o
simakovaalina@fedora:~/work/arch-pc/lab09$ gdb lab9-2
GNU gdb (Fedora Linux) 16.2-3.fc42
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab9-2...
(gdb)
```

Рис. 4.4: Запуск программы в отладчике

Запустив программу командой `run`, я убедилась в том, что она работает исправно (рис. 4.5).

```

simakovaalina@fedora:~/work/arch-pc/lab09$ touch lab9-2.asm
simakovaalina@fedora:~/work/arch-pc/lab09$ mousepad lab9-2.asm
simakovaalina@fedora:~/work/arch-pc/lab09$ nasm -f elf -g -l lab9-2.lst lab9-2.asm
simakovaalina@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab9-2 lab9-2.o
simakovaalina@fedora:~/work/arch-pc/lab09$ gdb lab9-2
GNU gdb (Fedora Linux) 16.2-3.fc42
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab9-2...
(gdb) run
Starting program: /home/simakovaalina/work/arch-pc/lab09/lab9-2

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.fedoraproject.org/>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading 51.96 K separate debug info for system-supplied DSO at 0xf7ffc000
Hello, world!
[Inferior 1 (process 4623) exited normally]
(gdb)

```

Рис. 4.5: Проверка программы отладчиком

Для более подробного анализа программы добавляю брейкпоинт на метку `_start` и снова запускаю отладку (рис. 4.6).

```

simakovaalina@fedora:~/work/arch-pc/lab09$ mousepad lab9-2.asm
simakovaalina@fedora:~/work/arch-pc/lab09$ nasm -f elf -g -l lab9-2.lst lab9-2.asm
simakovaalina@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab9-2 lab9-2.o
simakovaalina@fedora:~/work/arch-pc/lab09$ gdb lab9-2
GNU gdb (Fedora Linux) 16.2-3.fc42
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab9-2...
(gdb) run
Starting program: /home/simakovaalina/work/arch-pc/lab09/lab9-2

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.fedoraproject.org/>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading 51.96 K separate debug info for system-supplied DSO at 0xf7ffc000
Hello, world!
[Inferior 1 (process 4623) exited normally]
(gdb) break _start
Breakpoint 1 at 0x8048080: file lab9-2.asm, line 11.
(gdb) run
Starting program: /home/simakovaalina/work/arch-pc/lab09/lab9-2

Breakpoint 1, _start () at lab9-2.asm:11
11      mov eax, 4
(gdb)

```

Рис. 4.6: Запуск отладчика с брейкпоинтом

Далее смотрю дисассимилированный код программы, перевожу на команд с синтаксисом Intel амд топчик (рис. 4.7).

Различия между синтаксисом АТТ и Intel заключаются в порядке операндов (АТТ - Операнд источника указан первым. Intel - Операнд назначения указан первым), их размере (АТТ - размер операндов указывается явно с помощью суффиксов, непосредственные операнды предваряются символом \$; Intel – Размер операндов неявно определяется контекстом, как ax, eax, непосредственные операнды пишутся напрямую), именах регистров(АТТ - имена регистров предваряются символом %, Intel - имена регистров пишутся без префиксов).

```

Breakpoint 1, _start () at lab9-2.asm:11
11      mov eax, 4
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08048080 <+0>:  mov     $0x4,%eax
    0x08048085 <+5>:  mov     $0x1,%ebx
    0x0804808a <+10>: mov     $0x049000,%ecx
    0x0804808f <+15>: mov     $0x8,%edx
    0x08048094 <+20>: int     $0x80
    0x08048096 <+22>: mov     $0x4,%eax
    0x0804809b <+27>: mov     $0x1,%ebx
    0x080480a0 <+32>: mov     $0x049000,%ecx
    0x080480a5 <+37>: mov     $0x7,%edx
    0x080480aa <+42>: int     $0x80
    0x080480ac <+44>: mov     $0x1,%eax
    0x080480b1 <+49>: mov     $0x0,%ebx
    0x080480b6 <+54>: int     $0x80
End of assembler dump.
(gdb) set disassembly-flavor intel
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08048080 <+0>:  mov     eax,0x4
    0x08048085 <+5>:  mov     ebx,0x1
    0x0804808a <+10>: mov     ecx,0x8049000
    0x0804808f <+15>: mov     edx,0x8
    0x08048094 <+20>: int     0x80
    0x08048096 <+22>: mov     eax,0x4
    0x0804809b <+27>: mov     ebx,0x1
    0x080480a0 <+32>: mov     ecx,0x8049000
    0x080480a5 <+37>: mov     edx,0x7
    0x080480aa <+42>: int     0x80
    0x080480ac <+44>: mov     eax,0x1
    0x080480b1 <+49>: mov     ebx,0x0
    0x080480b6 <+54>: int     0x80
End of assembler dump.
(gdb)

```

Рис. 4.7: Дисассимилирование программы

Включаю режим псевдографики для более удобного анализа программы (рис. 4.8).

```

Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffcfa0 0xffffcfa0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048080 0x8048080 <_start>
eflags   0x202    [ IF ]
cs       0x23     35

lab9-2.asm
10 _start:
B> 11 mov eax, 4
    12 mov ebx, 1
    13 mov ecx, msg1
    14 mov edx, msg1Len
    15 int 0x80
    16 mov eax, 4
    17 mov ebx, 1
    18 mov ecx, msg2
    19 mov edx, msg2Len
    20 int 0x80

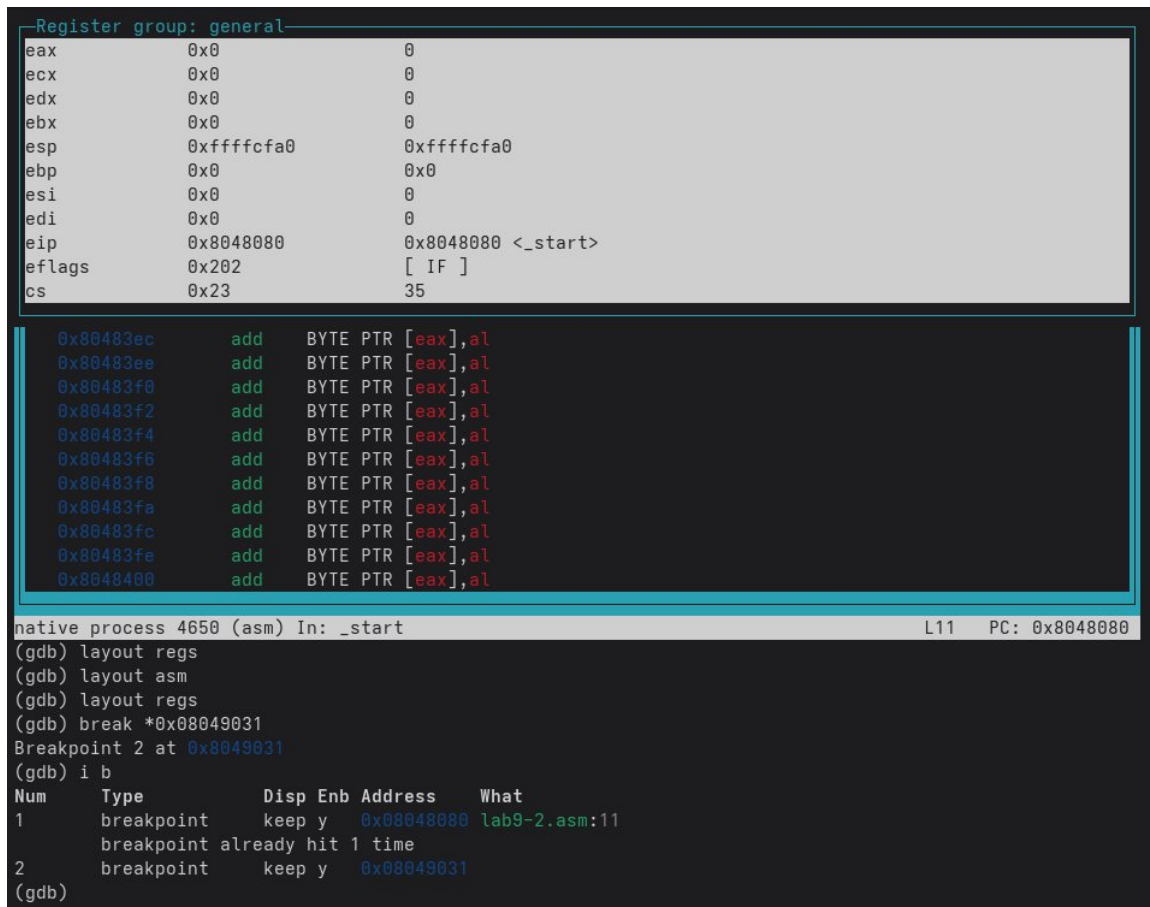
```

native process 4650 (src) In: _start L11 PC: 0x8048080
(gdb) layout regs
(gdb)

Рис. 4.8: Режим псевдографики

4.3 Добавление точек останова

Проверяю в режиме псевдографики, что брейкпоинт сохранился (рис. 4.9).



The screenshot shows a debugger interface with two main panels. The top panel, titled 'Register group: general', displays the values of various registers. The bottom panel shows a list of breakpoints.

Register	Value	Comment
eax	0x0	
ecx	0x0	
edx	0x0	
ebx	0x0	
esp	0xffffcfa0	0xffffcfa0
ebp	0x0	0x0
esi	0x0	
edi	0x0	
eip	0x8048080	0x8048080 <_start>
eflags	0x202	[IF]
cs	0x23	35

Address	Operation
0x80483ec	add BYTE PTR [eax],al
0x80483ee	add BYTE PTR [eax],al
0x80483f0	add BYTE PTR [eax],al
0x80483f2	add BYTE PTR [eax],al
0x80483f4	add BYTE PTR [eax],al
0x80483f6	add BYTE PTR [eax],al
0x80483f8	add BYTE PTR [eax],al
0x80483fa	add BYTE PTR [eax],al
0x80483fc	add BYTE PTR [eax],al
0x80483fe	add BYTE PTR [eax],al
0x8048400	add BYTE PTR [eax],al

native process 4650 (asm) In: _start L11 PC: 0x8048080

```
(gdb) layout regs
(gdb) layout asm
(gdb) layout regs
(gdb) break *0x08049031
Breakpoint 2 at 0x08049031
(gdb) i b
Num    Type           Disp Enb Address      What
1      breakpoint    keep y   0x08048080  lab9-2.asm:11
       breakpoint already hit 1 time
2      breakpoint    keep y   0x08049031
(gdb)
```

Рис. 4.9: Список брейкпоинтов

Усианавливаю ещё одну точку останова по адресу инструкции (рис. 4.10).

```
Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffcfa0 0xffffcfa0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048080 0x8048080 <_start>
eflags   0x202    [ IF ]
cs       0x23     35

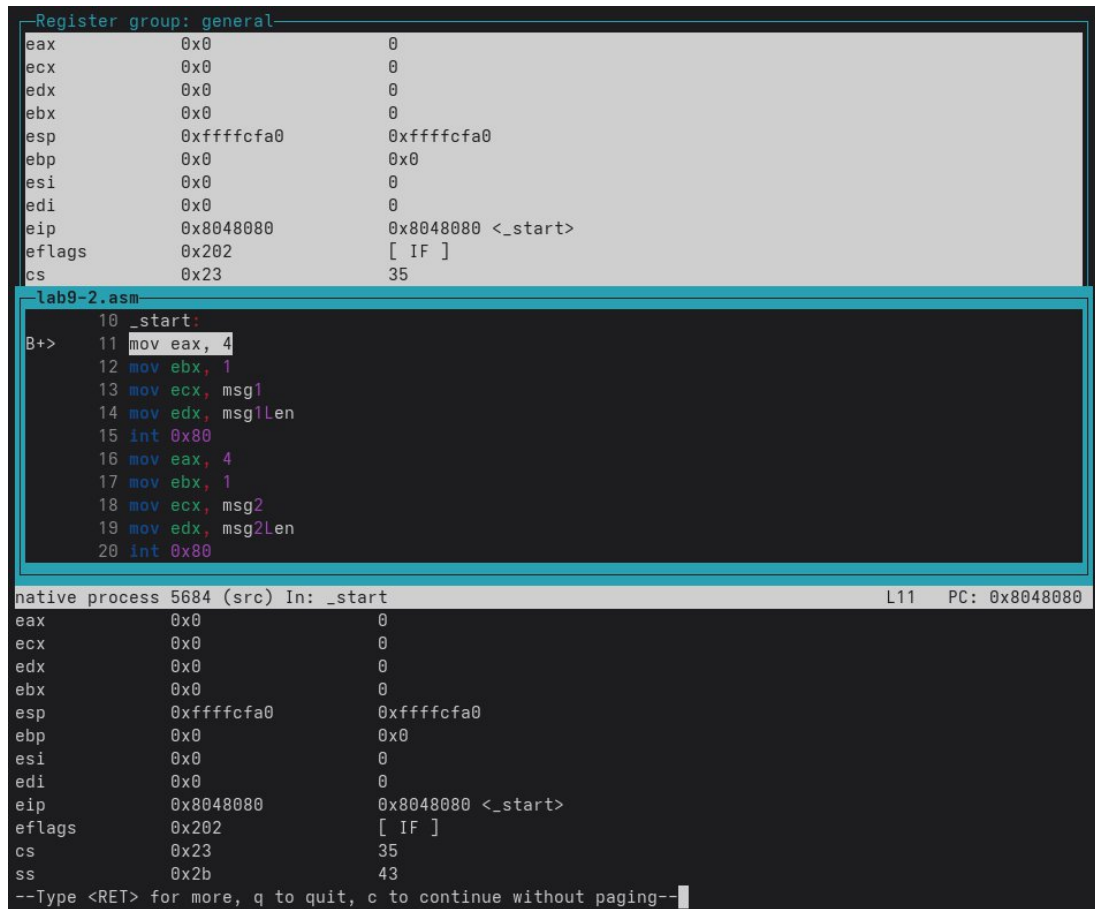
0x80483ec  add  BYTE PTR [eax],al
0x80483ee  add  BYTE PTR [eax],al
0x80483f0  add  BYTE PTR [eax],al
0x80483f2  add  BYTE PTR [eax],al
0x80483f4  add  BYTE PTR [eax],al
0x80483f6  add  BYTE PTR [eax],al
0x80483f8  add  BYTE PTR [eax],al
0x80483fa  add  BYTE PTR [eax],al
0x80483fc  add  BYTE PTR [eax],al
0x80483fe  add  BYTE PTR [eax],al
0x8048400  add  BYTE PTR [eax],al

native process 4650 (asm) In: _start L11 PC: 0x8048080
breakpoint already hit 1 time
3 breakpoint keep y 0x8048024
4 breakpoint keep y 0x8048094 lab9-2.asm:15
(gdb) delete 3
(gdb) delete 4
(gdb) break lab9-2.asm:15
Breakpoint 5 at 0x8048094: file lab9-2.asm, line 15.
(gdb) i b
Num    Type           Disp Enb Address      What
1      breakpoint      keep y 0x8048080 lab9-2.asm:11
      breakpoint already hit 1 time
5      breakpoint      keep y 0x8048094 lab9-2.asm:15
(gdb)
```

Рис. 4.10: Добавление второй точки останова

4.4 Работа с данными программы GBD

Просматриваю содержимое регистров командой `info registers` (рис. 4.11).



```
Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffcfa0 0xffffcfa0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048080 0x8048080 <_start>
eflags   0x202    [ IF ]
cs       0x23     35

lab9-2.asm
10 _start:
11 mov eax, 4
12 mov ebx, 1
13 mov ecx, msg1
14 mov edx, msg1Len
15 int 0x80
16 mov eax, 4
17 mov ebx, 1
18 mov ecx, msg2
19 mov edx, msg2Len
20 int 0x80

native process 5684 (src) In: _start L11 PC: 0x8048080
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffcfa0 0xffffcfa0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048080 0x8048080 <_start>
eflags   0x202    [ IF ]
cs       0x23     35
ss       0x2b     43
--Type <RET> for more, q to quit, c to continue without paging--
```

Рис. 4.11: Просмотр содержимого регистров

Смотрю содержимое переменных по имени и по адресу (рис. 4.12).

```
Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffcfa0 0xffffcfa0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048080 0x8048080 <_start>
eflags   0x202    [ IF ]
cs       0x23     35

1 SECTION .data
2 msg1: db "Hello, ",0x0
3 msg1len: equ $ - msg1
4 msg2: db "world!",0xa
5 msg2len: equ $ - msg2
6
7 SECTION .text
8 global _start
9
10 _start:
B+> 11 mov eax, 4

native process 5684 (src) In: _start L11 PC: 0x8048080
esi      0x0      0
edi      0x0      0
eip      0x8048080 0x8048080 <_start>
eflags   0x202    [ IF ]
cs       0x23     35
ss       0x2b     43
--Type <RET> for more, q to quit, c to continue without paging--q
Quit
(gdb) x/lsb &msg1
0x8049000 <msg1>: "Hello, "
(gdb) x/lsb &msg2
0x8049008 <msg2>: "world!\n\034"
(gdb)
```

Рис. 4.12: Просмотр содержимого переменных двумя способами
Меняю содержимое переменных по имени и по адресу (рис. 4.13).

```

Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffcfa0 0xffffcfa0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048080 0x8048080 <_start>
eflags   0x202    [ IF ]
cs       0x23     35

1 SECTION .data
2 msg1: db "Hello, ",0x0
3 msg1len: equ $ - msg1
4 msg2: db "world!",0xa
5 msg2len: equ $ - msg2
6
7 SECTION .text
8 global _start
9
10 _start:
B+> 11 mov eax, 4

native process 5684 (src) In: _start L11 PC: 0x8048080
--Type <RET> for more, q to quit, c to continue without paging--q
Quit
(gdb) x/lsb &msg1
0x8049000 <msg1>: "Hello, "
(gdb) x/lsb &msg2
0x8049008 <msg2>: "world!\n\034"
(gdb) set {char}&msg1 = 'h'
(gdb) x/lsb &msg1
0x8049000 <msg1>: "hello, "
(gdb) set {char}&msg2 = 'j'
(gdb) x/lsb &msg2
0x8049008 <msg2>: "jorld!\n\034"
(gdb)

```

Рис. 4.13: Изменение содержимого переменных двумя способами
Вывожу в различных форматах значение регистра edx (рис. 4.14).

```

Register group: general
eax      0x4      4
ecx      0x8049000 134516736
edx      0x8      8
ebx      0x1      1
esp      0xffffcfa0 0xffffcfa0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048094 0x8048094 <_start+20>
eflags   0x202    [ IF ]
cs       0x23     35

lab9-2.asm
B+ 11 mov eax, 4
    12 mov ebx, 1
    13 mov ecx, msg1
    14 mov edx, msg1len
> 15 int 0x80
    16 mov eax, 4
    17 mov ebx, 1
    18 mov ecx, msg2
    19 mov edx, msg2len
    20 int 0x80
    21 mov eax, 1

native process 5869 (src) In: _start L15 PC: 0x8048094
(gdb) p/s $edx
$13 = 8
(gdb) p/x $edx
$14 = 0x8
(gdb) p/t $edx
$15 = 1000
(gdb)

```

Рис. 4.14: Просмотр значение регистра разными представлениями

С помощью команды set меняю содержимое регистра ebx (рис. 4.15).

```
Register group: general
eax      0x4      4
ecx      0x8049000 134516736
edx      0x8      8
ebx      0x2      2
esp      0xffffcfa0 0xffffcfa0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048094 0x8048094 <_start+20>
eflags   0x202    [ IF ]
cs       0x23     35

lab9-2.asm
B+ 11 mov eax, 4
    12 mov ebx, 1
    13 mov ecx, msg1
    14 mov edx, msg1Len
> 15 int 0x80
    16 mov eax, 4
    17 mov ebx, 1
    18 mov ecx, msg2
    19 mov edx, msg2Len
    20 int 0x80
    21 mov eax, 1

native process 5869 (src) In: _start L15 PC: 0x8048094
(gdb) p/x $edx
$14 = 0x8
(gdb) p/t $edx
$15 = 1000
(gdb) set $ebx = '2'
(gdb) p/s
$16 = 8
(gdb) p/s $ebx
$17 = 50
(gdb) set $ebx = 2
(gdb) p/s $ebx
$18 = 2
(gdb)
```

Рис. 4.15: Примеры использования команды set

4.5 Обработка аргументов командной строки в GDB

Копирую программу из предыдущей лабораторной работы в текущий каталог и создаю исполняемый файл с файлом листинга и отладки (рис. 4.16).

```
simakovaalina@fedora:~/work/arch-pc/lab09$ cp ~/work/arch-pc/lab08/lab8-2.asm ~/work/arch-pc/lab09/lab9-3.asm
simakovaalina@fedora:~/work/arch-pc/lab09$ nasm -f elf -g -l lab9-3.lst lab9-3.asm
simakovaalina@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab9-3 lab9-3.o
simakovaalina@fedora:~/work/arch-pc/lab09$
```

Рис. 4.16: Подготовка новой программы

Запускаю программу в режиме отладки с указанием аргументов, указываю брейкпоинт и запускаю отладку. Проверяю работу стека, изменяя аргумент команды просмотра регистра esp на +4, число обусловлено разрядностью системы, а указатель void занимает как раз 4 байта, ошибка при аргументе +8 означает, что аргументы на вход программы закончились. (рис. 4.17).

```
simakovaalina@fedora:~/work/arch-pc/lab09$ cp ~/work/arch-pc/lab08/lab8-2.asm ~/work/arch-pc/lab09/lab9-3.asm
simakovaalina@fedora:~/work/arch-pc/lab09$ nasm -f elf -g -l lab9-3.lst lab9-3.asm
simakovaalina@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab9-3 lab9-3.o
simakovaalina@fedora:~/work/arch-pc/lab09$ gdb lab9-3
GNU gdb (Fedora Linux) 16.2-3.fc42
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab9-3...
(gdb) b _start
Breakpoint 1 at 0x8048148: file lab9-3.asm, line 7.
(gdb) run
Starting program: /home/simakovaalina/work/arch-pc/lab09/lab9-3

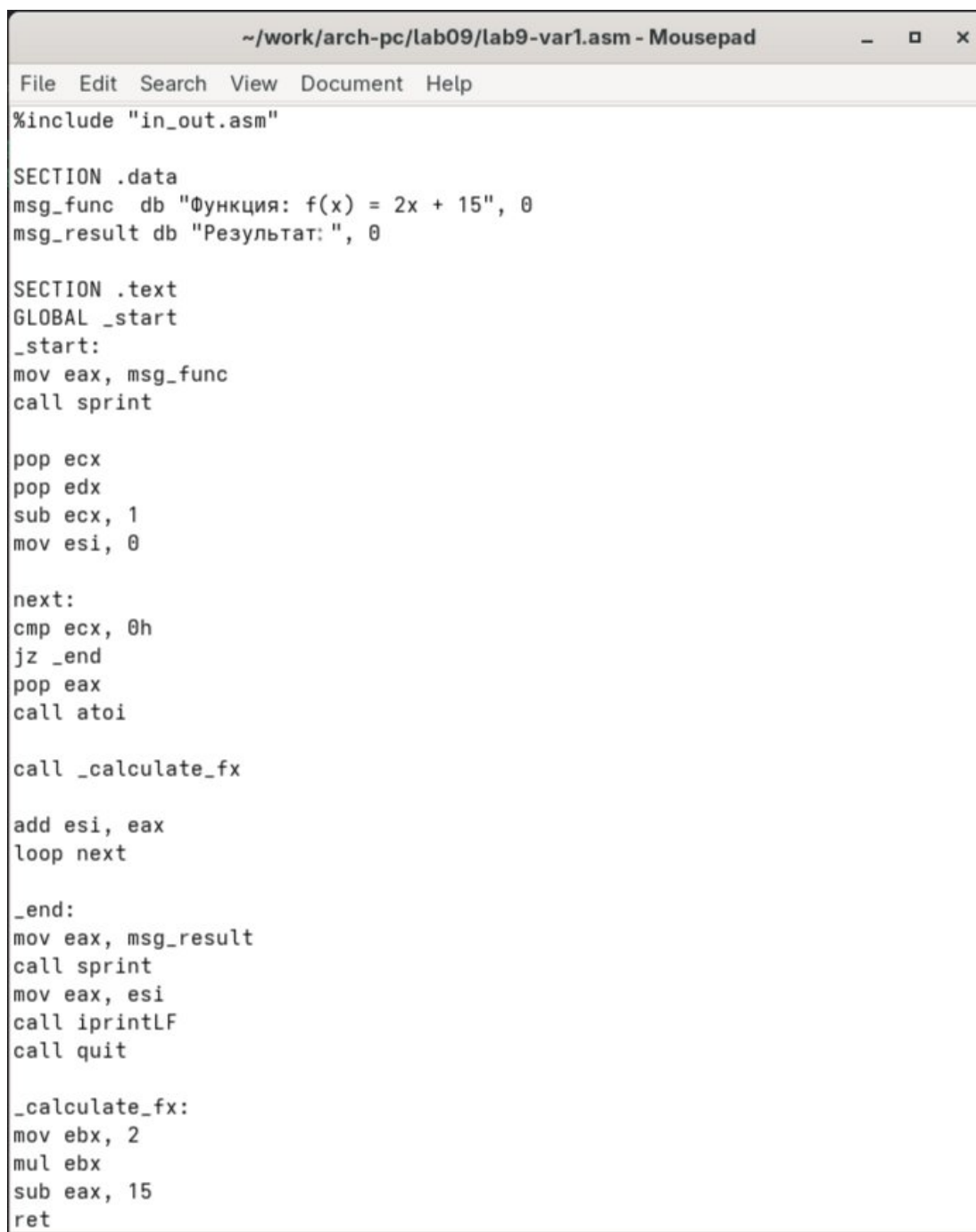
This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.fedoraproject.org/>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.

Breakpoint 1, _start () at lab9-3.asm:7
7       pop ecx
(gdb) x/s *(void**)(esp + 4)
0xffffd168:      "/home/simakovaalina/work/arch-pc/lab09/lab9-3"
(gdb) x/s *(void**)(esp + 8)
0x0:      <error: Cannot access memory at address 0x0>
(gdb)
```

Рис. 4.17: Проверка работы стека

5 Задания для самостоятельной работы

Преобразую программу самостоятельной части лабораторной работы №8 с использованием подпрограммы. (рис. 5.1).



```
~/work/arch-pc/lab09/lab9-var1.asm - Mousepad
File Edit Search View Document Help
%include "in_out.asm"

SECTION .data
msg_func db "Функция: f(x) = 2x + 15", 0
msg_result db "Результат: ", 0

SECTION .text
GLOBAL _start
_start:
mov eax, msg_func
call sprint

pop ecx
pop edx
sub ecx, 1
mov esi, 0

next:
cmp ecx, 0h
jz _end
pop eax
call atoi

call _calculate_fx

add esi, eax
loop next

_end:
mov eax, msg_result
call sprint
mov eax, esi
call iprintLF
call quit

_calculate_fx:
mov ebx, 2
mul ebx
sub eax, 15
ret
```

Рис. 5.1: Измененная программа лабораторной работы №8

Прикрепляю код программы:

```
%include "in_out.asm"
```

```
SECTION .data
```

```
msg_func db "Функция:  $f(x) = 2x + 15$ ", 0
msg_result db "Результат: ", 0
```

```
SECTION .text
```

```
GLOBAL _start
```

```
_start:
```

```
mov eax, msg_func
```

```
call sprint
```

```
pop ecx
```

```
pop edx
```

```
sub ecx, 1
```

```
mov esi, 0
```

```
next:
```

```
cmp ecx, 0h
```

```
jz _end
```

```
pop eax
```

```
call atoi
```

```
call _calculate_fx
```

```
add esi, eax
```

```
loop next
```

```
_end:
```

```
mov eax, msg_result
```

```
call sprint
```

```
mov eax, esi
```

```
call iprintLF
```

```
call quit
```

```

_calculate_fx:
mov ebx, 2
mul ebx
sub eax, 15
ret

```

Запускаю программу в режиме отладчика и пошагово через si просматриваю изменение значений регистров через i r. При выполнении инструкции mul esx можно заметить, что результат умножения записывается в регистр eax, но также меняет и edx. Значение регистра ebx не обновляется напрямую, поэтому результат программа неверно подсчитывает функцию (рис. 5.2).

```

simakovaalina@fedora:~/work/arch-pc/lab09 — gdb lab9-var1
~/work/arch-pc/lab09
simakovaalina@fedora:~/work/arch-pc/lab09 | simakovaalina@fedora:~/work/arch-pc/lab09 | simakovaalina@fedora:~/work/arch-pc/lab09
Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffcfa0 0xffffcfa0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048168 0x8048168 <_start>
eflags   0x202    [ IF ]
cs       0x23     35

0x8048156 <atoi.restore> pop %esi
0x8048157 <atoi.restore+1> pop %edx
0x8048158 <atoi.restore+2> pop %ecx
0x8048159 <atoi.restore+3> pop %ebx
0x804815a <atoi.restore+4> ret
0x804815b <quit> mov $0x0,%ebx
0x8048160 <quit+5> mov $0x1,%eax
0x8048165 <quit+10> int $0x80
0x8048167 <quit+12> ret
B>0x8048168 <_start> mov $0x8049000,%eax
0x804816d <_start+5> call 0x804800f <sprint>

native process 6778 (asm) In: _start L10 PC: 0x8048168
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffcfa0 0xffffcfa0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8048168 0x8048168 <_start>
eflags   0x202    [ IF ]
cs       0x23     35
ss       0x2b     43
--Type <RET> for more, q to quit, c to continue without paging--

```

Рис. 5.2: Поиск ошибки в программе через пошаговую отладку

Исправляю найденную ошибку, теперь программа верно считает значение функции (рис. 5.3).

```

simakovaalina@fedora:~/work/arch-pc/lab09$ mousepad lab9-vr1.asm
simakovaalina@fedora:~/work/arch-pc/lab09$ nasm -f elf lab9-vr1.asm
simakovaalina@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab9-vr1 lab9-vr1.o
simakovaalina@fedora:~/work/arch-pc/lab09$ ./lab9-vr1
Функция:  $f(x) = 2x + 15$ 
Результат: 0
simakovaalina@fedora:~/work/arch-pc/lab09$ ./lab9-vr1 3
Функция:  $f(x) = 2x + 15$ 
Результат: 21
simakovaalina@fedora:~/work/arch-pc/lab09$

```

Рис. 5.3: Проверка корректировок в программе

Прикрепляю код изменённой программы:

```
%include "in_out.asm"
```

```
SECTION .data
```

```
msg_func db "Функция:  $f(x) = 2x + 15$ ", 0
```

```
msg_result db "Результат: ", 0
```

```
SECTION .text
```

```
GLOBAL _start
```

```
_start:
```

```
mov eax, msg_func
```

```
call sprintLF
```

```
pop ecx
```

```
pop edx
```

```
sub ecx, 1
```

```
mov esi, 0
```

```
cmp ecx, 0
```

```
je _end
```

```
next:
```

```
pop eax
```

```
call atoi
```

```
call _calculate_fx
```

add esi, eax

dec ecx

jnz next

_end:

mov eax, msg_result

call sprint

mov eax, esi

call iprintLF

call quit

_calculate_fx:

mov ebx, 2

mul ebx

add eax, 15

ret

6 Выводы

При выполнении данной лабораторной работы я приобрела навыки написания программ с использованием подпрограмм, познакомилась с методами отладки при помощи GDB и его основными возможностями.

Список литературы

1. <https://esystem.rudn.ru/mod/page/view.php?id=1030492>
2. <https://esystem.rudn.ru/mod/resource/view.php?id=1030495>
3. https://esystem.rudn.ru/pluginfile.php/2089096/mod_resource/content/0/%D0%9B%D0%B0%D0%B1%D0%BE%D1%80%D0%B0%D1%82%D0%BE%D1%80%D0%BD%D0%B0%D1%8F%20%D1%80%D0%B0%D0%B1%D0%BE%D1%82%D0%B0%20%E2%84%969.%20%D0%9F%D0%BE%D0%BD%D1%8F%D1%82%D0%B8%D0%B5%20%D0%BF%D0%BE%D0%B4%D0%BF%D1%80%D0%BE%D0%B3%D1%80%D0%B0%D0%BC%D0%BC%D1%8B.%20%D0%9E%D1%82%D0%BB%D0%B0%D0%B4%D1%87%D0%B8%D0%BA%20..pdf
4. <https://esystem.rudn.ru/mod/resource/view.php?id=1030496>